

Sparse-Graph Representations for Scalable Grouped Fused Regression

Daniel Agyapong

*Northern Arizona University
Flagstaff, AZ, USA*

DA2343@NAU.EDU

Mike Gowanlock

*School of Informatics, Computing, and Cyber Systems
Northern Arizona University
Flagstaff, AZ, USA*

Jane Marks

*Department of Biological Sciences
Northern Arizona University
Flagstaff, AZ, USA*

Toby Dylan Hocking

*Département d'informatique
Université de Sherbrooke
Sherbrooke, QC, Canada*

Editor:

Abstract

Grouped regression models estimate related linear parameters for each group by combining sparsity with fusion penalties across a graph of groups. Although the statistical graph is often sparse, standard implementations can still materialize full pairwise fusion operators or augmented designs, making the computational representation much denser than the model. This paper studies sparse-graph representations for grouped fused regression and shows that the graph edge set can be preserved directly in the optimizer. For L1 fusion, an active-edge operator evaluates the same smoothed proximal-gradient fusion term using only nonzero graph edges. For L2 fusion, an active-edge augmented-design construction emits sparse triplets only for the modeled edges before solving the regularized path. These constructions preserve the legacy objective in the exact L1 and L2 cases while reducing the dominant fusion representation from all-pairs graph construction to edge-linear construction. The empirical comparisons intentionally hold the optimizer family fixed relative to the legacy grouped fused-regression solvers, so that the measured differences isolate graph representation rather than changes in optimization architecture. An L1 chain approximation is also analyzed as a deliberate approximation that trades graph fidelity for additional efficiency. Across public grouped-regression benchmarks, the exact L1 operator matches legacy predictive accuracy while reducing median runtime by 2.26x and median peak memory by 10.7%. The chain variant gives a median 3.26x runtime reduction with an accuracy tradeoff. Across 19 L2 benchmark datasets, active-edge construction has no systematic loss in test error and reduces median fit time by 14.9x. The algorithms are implemented in the open-source R package `sparsefusion`, with benchmark scripts and summaries provided for reproducibility.

Keywords: fused lasso, grouped regression, sparse graphs, structured sparsity, first-order optimization

1 Introduction

Many regression problems contain a natural grouping variable: patients may belong to disease subtypes, schools to districts, countries to regions, or time series to meters. Fitting a separate model in each group ignores shared signal, whereas fitting one global model ignores differences across groups. Grouped fused regression addresses this middle ground by estimating one coefficient vector for each group and penalizing differences between coefficient vectors along a graph of group relationships.

Let $X_g \in \mathbb{R}^{n_g \times p}$ and $y_g \in \mathbb{R}^{n_g}$ denote the data for group g , and let $\beta_g \in \mathbb{R}^p$ be that group's linear coefficient vector. For a graph $G = (V, E)$ over k groups with nonnegative edge weights w_{gh} , the estimators considered in this paper have the form

$$\min_{\beta_1, \dots, \beta_k} \sum_{g=1}^k \frac{1}{2n_g} \|y_g - X_g \beta_g\|_2^2 + \lambda \sum_{g=1}^k \|\beta_g\|_1 + \gamma \sum_{(g,h) \in E} w_{gh} \Omega(\beta_g - \beta_h). \quad (1)$$

For the L1 fused model, $\Omega(u) = \|u\|_1$; for the L2 fused model, $\Omega(u) = \frac{1}{2} \|u\|_2^2$ after the standard augmented-design reformulation. Optional group-specific intercepts are included in the software implementation but are excluded from sparsity and fusion penalties. This formulation is a direct descendant of the lasso, fused lasso, grouped and elastic-net regularization, graph-structured fused optimization, and subgroup regression models reviewed below. The L1 solver uses the standard Nesterov smoothing strategy for nonsmooth penalties (Nesterov, 2005).

The central computational issue is a mismatch between the statistical object and the solver representation. A complete graph has $k(k-1)/2$ edges, but practical graphs are frequently chains, trees, nearest-neighbor graphs, or domain-informed sparse networks. If the objective is defined on only m active edges, a solver that builds all pairwise fusion constraints and then uses zero weights for inactive edges changes the computational scaling of the problem without changing its statistical content. This matters as k grows: the modeling assumption is sparse, but the legacy representation is still quadratic.

The paper therefore focuses on a controlled algorithmic comparison rather than on replacing the optimizer family. For L1 fusion, both the legacy solver and the exact active-edge solver use the same smoothed proximal-gradient framework. For L2 fusion, both the legacy solver and the active-edge solver use the same augmented-design reduction before fitting the regularized path. Alternative methods such as ADMM and split-Bregman solvers are important for nonsmooth fused penalties, but they change the optimization architecture and introduce additional tuning and implementation choices. The comparisons here are designed to isolate the effect of preserving the sparse graph inside the existing grouped fused-regression optimization framework.

The contribution of this paper is an algorithmic representation of grouped fused regression that makes the sparse graph a first-class object in the optimizer. The key idea is to represent the fused-regression objective in a way that respects the graph on which the penalty is defined. First, the paper gives an exact active-edge L1 operator form that evaluates the smoothed proximal-gradient fusion term only over active edges. Second, it gives

an exact active-edge L2 augmented-design construction that emits sparse triplets only for nonzero graph edges before delegating the regularized regression path to `glmnet` (Friedman et al., 2010). Third, it separates these exact reformulations from approximation-oriented solver modes, including an L1 chain approximation that collapses a graph to $k - 1$ chain edges using a depth-first traversal, motivated by chain-structured fused lasso solvers. The result is a reusable design pattern for multi-task fused regression and other graph-penalized models whose penalties decompose over graph edges.

The main contributions are:

- a sparse-graph representation principle for grouped fused regression solvers;
- exact active-edge L1 and L2 reformulations that preserve the legacy objective while avoiding zero-weight all-pair construction;
- a clear separation between exact sparse-graph solvers and approximate high-efficiency chain variants;
- a controlled empirical design that keeps the optimizer family fixed and isolates the effect of graph representation;
- empirical comparisons showing when fused regression improves over unfused `glmnet` baselines and when active-edge construction improves over legacy fused solvers; and
- a reproducible open-source implementation with benchmark scripts, manifests, and figure-generation artifacts.

The algorithms are implemented in the R package `sparsefusion`. The current benchmark suite uses public grouped regression datasets and synthetic scaling experiments to compare prediction error, wall time, and peak resident memory. The results show that graph-aware construction can deliver substantial runtime and memory reductions without changing the statistical objective for the exact operator and L2 variants.

The remainder of the paper is organized as follows. Section 2 reviews fused and graph-structured regression methods, together with related optimization approaches. Section 3 defines the grouped fused-regression model and the legacy full-pairwise computations used as references. Section 4 presents the active-edge L1 operator, the active-edge L2 augmented-design construction, and the chain approximation. Section 5 describes the benchmark protocol, datasets, and compared methods. Section 6 reports accuracy, timing, memory, and Pareto efficiency results. Section 7 discusses implications, limitations, and implementation guidance, and Section 8 summarizes reproducibility materials.

2 Related Work

The fused lasso extends the lasso by combining sparsity with penalties on ordered coefficient differences (Tibshirani, 1996; Tibshirani et al., 2005). This places grouped fused regression in a broader family of structured regularizers, including the elastic net, group lasso, generalized lasso, and graph trend filtering (Zou and Hastie, 2005; Yuan and Lin, 2006; Tibshirani and Taylor, 2011; Wang et al., 2016). Specialized algorithms for one-dimensional signal approximation, including path algorithms for the fused lasso signal approximator, exploit

the ordered chain structure of that problem (Hoeffling, 2010). Split Bregman and ADMM methods provide another important optimization line for nonsmooth fused penalties (Ye and Xie, 2011; Boyd et al., 2011). General graph fusion is more flexible but computationally harder because the penalty couples many coefficient blocks.

It is useful to separate three forms of fusion that are sometimes discussed under related fused-lasso terminology. In the classic fused lasso, the ordered objects being fused are usually variables, coordinates, or signal locations. For a single regression vector, this means penalizing adjacent coefficient differences such as $|\beta_{j+1} - \beta_j|$. In graph-guided multi-task learning, the fused objects are tasks, often corresponding to related responses or prediction problems. In grouped fused regression, the fused objects are groups of observations in one regression problem. The model estimates a coefficient matrix $B \in \mathbb{R}^{p \times k}$, with one column for each group. The fusion graph is on the groups, and an edge (g, h) penalizes the difference between the two coefficient vectors B_g and B_h . Thus the paper studies fusion between groups rather than fusion between variables or responses.

Figure 1 illustrates these distinctions. Classic variable fusion acts within a single ordered coefficient vector, as in the fused lasso (Tibshirani et al., 2005). Multi-task fusion encourages sharing across columns of a coefficient matrix, where the columns correspond to responses or tasks (Chen et al., 2010, 2012a). Grouped fused regression also encourages sharing across columns, but the columns are groups from one grouped regression problem with common predictors and a common response (Dondelinger et al., 2020). This is why graph-guided multi-task learning is related but not identical to the setting studied here.

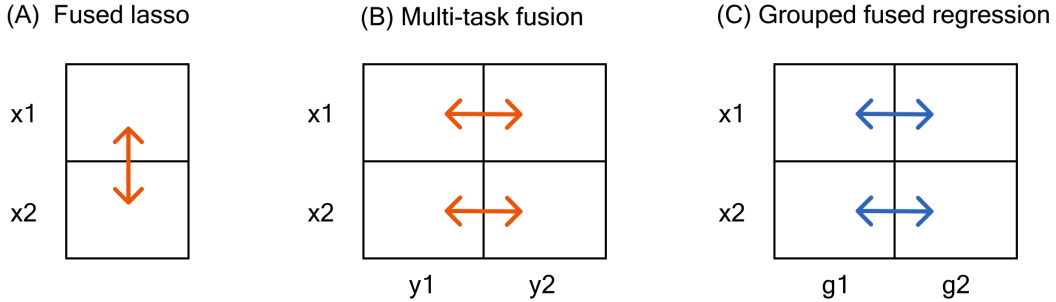


Figure 1: Three forms of fusion shown as coefficient-vector and coefficient-matrix schematics. Rows labeled x_1 and x_2 denote predictors. In Panel B, columns y_1 and y_2 denote related responses or tasks. In Panel C, columns g_1 and g_2 denote groups of observations. Colored links indicate which objects are coupled by the fusion penalty. Panel A shows fused lasso penalties on ordered coefficients (Tibshirani et al., 2005). Panel B shows graph-guided multi-task fusion across related responses or tasks (Chen et al., 2010, 2012a). Panel C shows grouped fused regression, where the fusion graph is placed on groups of observations (Dondelinger et al., 2020).

Graph-structured multi-task regression extended these ideas to arbitrary task graphs and developed smoothed proximal-gradient methods for the general fused lasso (Chen et al.,

2010, 2012a). Related multi-task and scientific applications use graph-guided fusion for eQTL mapping, additive modeling, gene regulatory network inference, and grouped regression (Chen et al., 2012b; Petersen et al., 2016; Omranian et al., 2016). Network lasso methods (Hallac et al., 2015), graph trend filtering, DFS fused lasso denoising (Tansey et al., 2018), and k-nearest-neighbor fused lasso estimators (Padilla et al., 2020) further emphasize that the graph itself is part of the modeling problem: it may come from prior knowledge, distances between groups, or other data-driven structure.

The present work focuses on how grouped fused regression objectives should be represented and solved when the graph is sparse. This is more than an implementation detail: the graph is part of the modeling assumption, and a dense all-pairs representation discards that assumption at the computational layer. The active-edge constructions developed below restore the intended edge-level representation while preserving the legacy objective in the exact L1 and L2 cases.

Table 1 positions the paper relative to the main modeling and optimization ideas in the literature, rather than comparing software packages. Prior work supplies the sparsity and fusion penalties, general graph-difference formulations, data-driven graph construction, and smoothed first-order optimization. The present paper keeps the grouped fused-regression objective and optimizer family fixed, and studies sparse edge representations for L1 and L2 fusion.

Figures 2 and 3 summarize the main computational distinction between the legacy fused-regression representation and the sparse-graph representation used in `sparsefusion`. In both cases the statistical model is defined by a graph on the k groups, with an edge weight indicating how strongly two subgroup coefficient vectors should be encouraged to agree. The legacy construction first expands this modeling graph into a dense algebraic object over many group pairs. That expansion is benign for small k , but it becomes the dominant cost when the intended graph is sparse, because computation is then driven by the number of possible group pairs rather than by the number of modeled edges.

Figure 2 describes the L1 fusion case. Panel A begins with the intended fusion graph: nodes are regression groups and each drawn edge marks a coefficient difference penalized by the model. The legacy full-edge construction expands that graph into an all-pairs layout for the combined L1 constraint matrix; in the four-group sketch, it introduces pair slots that are absent from the modeled graph. The exact active-edge operator keeps the original graph and evaluates the nonsmooth fusion penalty only along its m nonzero edges. The chain approximation makes a different tradeoff by replacing the original edge set with a chain before fitting. Panel B then shows the computational consequence: the active-edge L1 representation avoids the all-pairs constraint matrix, so fusion-side scaling depends on the graph edges rather than powers of k .

Figure 3 gives the analogous picture for L2 fusion. In the quadratic case, fusion can be represented by augmenting the regression design with pseudo-observations that penalize differences between subgroup coefficients. The legacy construction emits augmented rows for the expanded set of group pairs; the active-edge construction emits sparse triplets only for edges with nonzero fusion weight. Thus the same modeling graph produces a much smaller linear-algebra problem whenever $m \ll k^2$. These two figures motivate the solver descriptions in Section 4 and the scaling experiments reported below: the central question

Table 1: Existing methodological literature and distinctions from the present work. The paper builds on graph-fused regression and smoothed first-order optimization, but focuses on preserving sparse edge sets in the solver representation while holding the objective and optimizer family fixed.

Literature	Main idea	Distinction from this paper
Lasso and fused lasso (Tibshirani, 1996; Tibshirani et al., 2005)	Combines sparsity with penalties on adjacent coefficient differences.	Introduces the core sparsity and fusion penalties, while this paper studies how to represent sparse fusion graphs efficiently in grouped fused-regression.
Graph-guided multitask regression (Chen et al., 2010, 2012a,b)	Uses task graphs and smoothed proximal-gradient methods for general fused multitask penalties.	Uses related graph fusion across columns of a coefficient matrix. In this paper the columns are groups from one grouped regression problem, and the focus is efficient sparse edge representation for L1 and L2 fusion.
Generalized lasso and graph trend filtering (Tibshirani and Taylor, 2011; Wang et al., 2016)	Uses linear operators or graph differences to encode structured variation.	Provides a broad operator framework, but does not isolate active-edge construction for paired L1/L2 grouped fused-regression solvers.
Split-Bregman and ADMM approaches (Ye and Xie, 2011; Boyd et al., 2011)	Introduce auxiliary variables for nonsmooth penalties and solve alternating subproblems.	The comparisons in this paper keep the smoothed first-order optimization framework fixed to isolate the effect of graph representation.
Network lasso and graph-based fusion (Hallac et al., 2015)	Penalizes graph-edge differences to cluster or share information over networks.	Uses graph-edge penalties to share information over networks, but studies a different objective and optimization framework from the grouped fused-regression in this paper.
Joint lasso for subgroup regression (Dondelinger et al., 2020)	Estimates grouped regressions with sparsity and fusion across groups.	Closest statistical predecessor and reference used in this paper. This paper preserves its objective while replacing all-pair graph construction with active-edge construction.
Graph decomposition for graph-fused lasso (Yu et al., 2025)	Uses graph decomposition to build a faster ADMM algorithm for graph-fused lasso on general graphs.	Changes the ADMM decomposition and optimizer architecture; this paper keeps the smoothed L1 and augmented-design L2 solver families fixed to measure representation effects.

is whether preserving the edge-level graph representation yields the same fitted objective at lower runtime and memory cost.

3 Model and Legacy Computation

The L1 solver in `sparsefusion` follows a smoothed proximal-gradient strategy for Equation 1. Let $B = [\beta_1, \dots, \beta_k]$ be the $p \times k$ coefficient matrix. The legacy implementation builds a constraint matrix with columns for sparsity and fusion constraints, then uses a smoothed dual approximation to compute the gradient contribution of the nonsmooth penalty. The

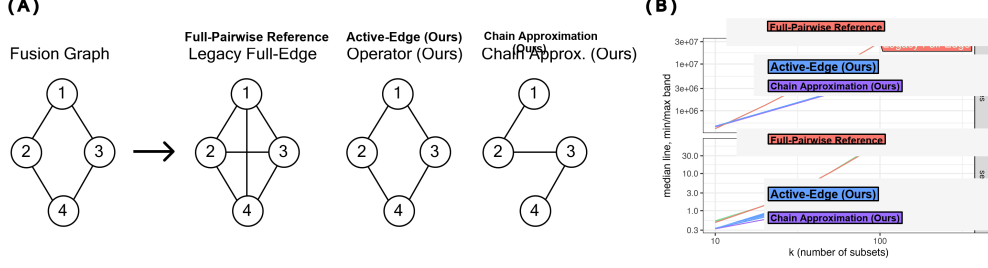


Figure 2: Conceptual overview of L1 sparse-graph fusion. A: fusion graph construction. The legacy full-edge construction associated with Dondelinger et al. (2020) forms a combined constraint matrix over all group pairs, while the active-edge operator works directly on the sparse graph. The chain approximation replaces the modeled edge set with a chain before fitting. B: asymptotic complexity. The synthetic example fixes $p = 120$ and $n_{\text{group,train}} = 35$, so at $k = 100$ the total training sample size is $n = 3500$. With a sparse chain graph and fixed remaining hyperparameters, the active-edge representation changes the dominant fusion cost from $O(pk^3)$ to $O(pm)$; the chain approximation is plotted alongside the exact operator as the chain-based L1 variant.

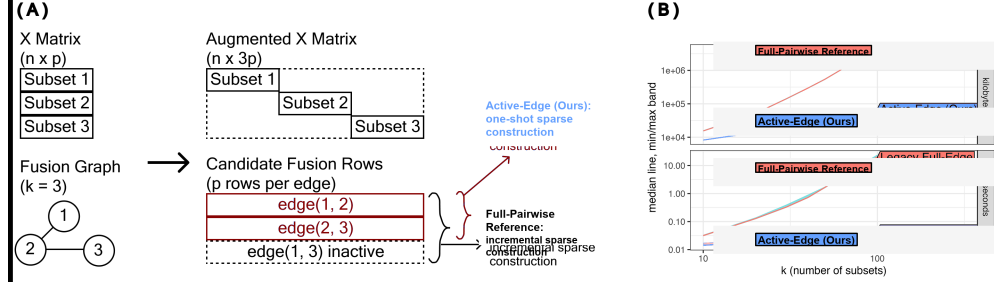


Figure 3: Conceptual overview of L2 sparse-graph fusion. A: augmented matrix construction. The legacy full-edge construction associated with Dondelinger et al. (2020) forms augmented rows for all group pairs, while the active-edge builder emits only sparse triplets for graph edges with nonzero weight. B: asymptotic complexity. The synthetic example fixes $p = 120$ and $n_{\text{group,train}} = 35$, so at $k = 100$ the total training sample size is $n = 3500$. With a sparse chain graph and fixed remaining hyperparameters, active-edge construction changes the dominant fusion-block cost from $O(pk^2)$ to $O(pm)$.

update has the generic accelerated form

$$A^* = \Pi_{[-1,1]} \left(\frac{BC}{\mu} \right), \quad (2)$$

$$\nabla f_{\mu}(B) = \nabla \ell(B) + A^* C^T, \quad (3)$$

$$B^{t+1} = W^t - L^{-1} \nabla f_{\mu}(B^t), \quad (4)$$

where C encodes the sparsity and fusion operators, μ is the smoothing parameter, L is a Lipschitz bound, and W^t is the accelerated iterate. When C contains all group pairs, storage and multiplication include $O(pk^2)$ fusion terms for a complete representation.

More explicitly, the L1 implementation uses the dual identity $|z| = \max_{|a| \leq 1} az$ and replaces the nonsmooth absolute-value terms by clipped smoothed duals. For sparsity and active fusion edges this gives

$$A_s^* = \text{clip} \left(\frac{\lambda B_s}{\mu}, -1, 1 \right), \quad A_{uv}^* = \text{clip} \left(\frac{\gamma w_{uv}(\beta_u - \beta_v)}{\mu}, -1, 1 \right),$$

where B_s excludes intercept rows. The code uses the step-size upper bound

$$L_U = \lambda_{\max} + \frac{\lambda^2 + 2\gamma^2 d_{\max}^{(2)}}{\mu},$$

with $\lambda_{\max} = \sum_g \lambda_{\max}(X_g^\top X_g / n_g)$ under scaled group losses and $d_{\max}^{(2)}$ any upper bound on $\max_g \sum_{h:(g,h) \in E} w_{gh}^2$ for the active fusion graph. This quantity is bounded by the usual weighted degree when weights are normalized to lie in $[0, 1]$.

3.1 Optimization Guarantee for Smoothed L1

The guarantees below are deterministic statements for a fixed graph, fixed hyperparameters, and fixed train/validation split. They concern objective representation and optimization, not statistical recovery under a data generating model. The first guarantee specializes the standard consequences of Nesterov smoothing and accelerated proximal-gradient convergence to the grouped fused-regression notation used here (Nesterov, 2005; Beck and Teboulle, 2009).

Proposition 1 (Lipschitz and convergence guarantee for smoothed L1)

Let $F_\mu(B)$ be the Nesterov-smoothed L1 grouped fused objective, before final postprocessing. Let B_μ^* minimize F_μ . If the APG update uses any L_U satisfying

$$L_U \geq \lambda_{\max} + \frac{\lambda^2 + 2\gamma^2 d_{\max}^{(2)}}{\mu},$$

then ∇F_μ is L_U -Lipschitz and the accelerated iterates satisfy

$$F_\mu(B^{(t)}) - F_\mu(B_\mu^*) \leq \frac{2L_U \|B^{(0)} - B_\mu^*\|_F^2}{(t+1)^2}.$$

Moreover, if N_c is the number of scalar sparsity and fusion constraints in the active-edge representation, then the nonsmoothed objective F and its smoothed version obey

$$0 \leq F(B) - F_\mu(B) \leq \frac{\mu N_c}{2}$$

for every coefficient matrix B .

Proof The least-squares part has Lipschitz constant bounded by $\lambda_{\max}$ under the scaled group losses. The smoothed sparsity operator contributes at most λ^2/μ to the gradient Lipschitz constant. For the fusion operator, each edge difference has two incident group blocks; the squared operator norm is bounded by $2d_{\max}^{(2)}$, giving the fusion contribution $2\gamma^2 d_{\max}^{(2)}/\mu$. Summing these bounds gives L_U . The stated $O(t^{-2})$ rate is the standard accelerated-gradient guarantee for convex differentiable objectives with L_U -Lipschitz gradient. Finally, each scalar absolute-value term satisfies $0 \leq |z| - \max_{|a| \leq 1} \{az - \mu a^2/2\} \leq \mu/2$; summing over the N_c scalar constraints gives the smoothing gap. ■

Proposition 2 (Equivalence of full-edge and active-edge L1 updates)

Fix a weighted graph G and let E denote the edges with positive weight. For smoothing parameter $\mu > 0$, suppose the dense smoothed proximal-gradient implementation includes one fusion column for every pair but assigns zero weight to inactive pairs. The active-edge operator implementation computes exactly the same smoothed fusion gradient as the dense implementation after removing the zero-weight columns.

Proof Let e_u be the u th coordinate vector in $\mathbb{R}^k$. For any pair $1 \leq u < v \leq k$, define the smoothed dual block

$$P_{uv}(B) = \text{clip} \left(\frac{\gamma w_{uv}(\beta_u - \beta_v)}{\mu}, -1, 1 \right).$$

The dense fusion contribution to the smoothed gradient can be written as

$$\Delta_{\text{dense}}(B) = \sum_{1 \leq u < v \leq k} \gamma w_{uv} P_{uv}(B) (e_u - e_v)^\top.$$

If $w_{uv} = 0$, then the corresponding summand is zero. Therefore

$$\Delta_{\text{dense}}(B) = \sum_{(u,v) \in E} \gamma w_{uv} P_{uv}(B) (e_u - e_v)^\top = \Delta_{\text{active}}(B).$$

Algorithm 2 computes this last expression by adding $\gamma w_{uv} P_{uv}(B)$ to column u and subtracting it from column v for each active edge. Hence the dense and active-edge implementations produce the same smoothed fusion-gradient contribution, and therefore the same L1 update when the remaining APG quantities are held fixed. ■

3.2 Full-Edge L2 Augmented Design

The L2 solver uses a different reduction. It constructs an augmented response and design matrix,

$$\tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix}, \quad \tilde{X} = \begin{bmatrix} X_{\text{block}} \\ X_{\text{fusion}} \end{bmatrix},$$

where X_{block} is block diagonal over groups and X_{fusion} contains, for each group pair, rows proportional to

$$\sqrt{\gamma w_{gh}}(\beta_g - \beta_h).$$

Algorithm 1 Full-Pairwise Smoothed APG Reference (old.11)

Require: grouped data, $G, \lambda, \gamma, \mu, T, \varepsilon$
 1: Construct full fusion matrix $C_{\text{fusion}} \in \mathbb{R}^{k \times \binom{k}{2}}$
 2: Construct $C = [\lambda I_k, \gamma C_{\text{fusion}}]$
 3: Initialize $B^{(0)} = 0, W^{(0)} = 0, S^{(0)} = 0$
 4: **for** $t = 1, \dots, T$ **do**
 5: Compute $\nabla \ell(B^{(t-1)})$ groupwise
 6: $A^* \leftarrow \text{clip}([B_s^{(t-1)} C_s, B_f^{(t-1)} C_f] / \mu, -1, 1)$
 7: $\nabla r_\mu(B^{(t-1)}) \leftarrow A^* C^\top$
 8: $H_t \leftarrow \nabla \ell(B^{(t-1)}) + \nabla r_\mu(B^{(t-1)})$
 9: Compute L_U from the dense max-degree proxy
 10: Update $\tilde{B}^{(t)}, S^{(t)}, W^{(t)}$
 11: **if** $\|\tilde{B}^{(t)} - B^{(t-1)}\|_1 < \varepsilon p$ **then**
 12: **break**
 13: **end if**
 14: $B^{(t)} \leftarrow \tilde{B}^{(t)}$
 15: **end for**
 16: Soft-threshold non-intercept rows by λ/n_g
 17: **return** $\hat{B}$

Let p_{eff} denote the number of coefficient coordinates included in this L2 fusion block after applying the intercept convention. The resulting lasso problem is passed to **glmnet**. A full all-pairs construction creates $O(p_{\text{eff}} k^2)$ fusion rows even when only $m = |E|$ edges are active.

Proposition 3 (Equivalence of active-edge L2 augmented rows) *For a fixed graph G , penalty γ , and scaling convention, the active-edge L2 augmented design is equivalent to the legacy all-pairs augmented design after deleting all zero-weight pair rows and applying a row permutation.*

Proof Each nonzero edge (u, v) contributes p_{eff} augmented rows with $+\sqrt{\gamma w_{uv}}$ in coefficient block u and $-\sqrt{\gamma w_{uv}}$ in coefficient block v , with zero augmented response. These rows contribute

$$\gamma w_{uv} \|\beta_u - \beta_v\|_2^2$$

to the augmented least-squares objective. Therefore the all-pairs fusion contribution satisfies

$$\sum_{1 \leq u < v \leq k} \gamma w_{uv} \|\beta_u - \beta_v\|_2^2 = \sum_{(u,v) \in E} \gamma w_{uv} \|\beta_u - \beta_v\|_2^2,$$

because inactive pairs have $w_{uv} = 0$. The active-edge builder constructs exactly the rows corresponding to the right-hand side using sparse triplets. Permuting those rows changes only their order, not the sum of squared residuals, so the objective minimized by **glmnet** is unchanged. ■

Proposition 4 (Sparse-graph construction cost)

Let k be the number of groups, p the number of penalized coefficients per group in the L1 fusion operator, $m = |E|$ the number of nonzero fusion edges, and $q = \binom{k}{2}$ the number of all possible group pairs. Let p_{eff} be the number of coefficients penalized by the L2 fusion block after the intercept convention is applied. In one L1 smoothed-gradient evaluation, a dense all-pairs operator performs fusion work proportional to pq after construction, whereas the active-edge operator performs fusion work proportional to pm . In the L2 augmented-design construction, the fusion block decreases from $p_{\text{eff}}q$ augmented rows to $p_{\text{eff}}m$ augmented rows.

Proof For L1 fusion, each represented edge requires a p -dimensional coefficient difference and a p -dimensional scatter back to the incident groups. Thus the fusion-side work scales as

$$W_{\text{L1,full}} \asymp pq = p \binom{k}{2}, \quad W_{\text{L1,active}} \asymp pm.$$

The dense all-pairs representation iterates over q candidate pairs, including zero-weight inactive pairs, while the active-edge representation iterates only over the m nonzero graph edges.

For L2 fusion, each represented edge contributes p_{eff} augmented rows, one for each penalized coefficient coordinate. Therefore the fusion block sizes are

$$R_{\text{L2,full}} = p_{\text{eff}}q = p_{\text{eff}} \binom{k}{2}, \quad R_{\text{L2,active}} = p_{\text{eff}}m.$$

Deleting zero-weight pairs gives the stated row reduction, with p_{eff} determined by the intercept convention. ■

4 Sparse-Graph Solvers

4.1 Exact L1 Operator Solver

Here $B = [\beta_1, \dots, \beta_k]$ is the grouped coefficient matrix, with one column for each group. The exact L1 operator solver replaces the dense fusion matrix representation with edge lists $(u_e, v_e, w_e)_{e=1}^m$, where $m = |E|$ is the number of active fusion edges, u_e and v_e are the incident groups for edge e , and $w_e > 0$ is its fusion weight. At each iteration, the solver computes the fusion differences only on active edges,

$$D_e = \beta_{u_e} - \beta_{v_e}, \quad P_e = \text{clip} \left(\frac{\gamma w_e D_e}{\mu}, -1, 1 \right),$$

where D_e is the coefficient difference vector and P_e is its clipped smoothed dual block. The solver scatters $\gamma w_e P_e$ back to the incident group columns with opposite signs. The sparsity part of the update remains unchanged. This representation has the same objective and first-order updates as the active-edge version of the legacy smoothed proximal-gradient solver, but the fusion-gradient work scales with pm rather than pk^2 .

The implementation also supports block processing of edges. This is a practical memory safeguard for large p or m , since intermediate edge-difference matrices can otherwise dominate the working set. The operator solver is therefore intended as the general exact path for sparse graphs.

Active-edge gradient assembly. The operator implementation initializes $\Delta_f = 0$ and loops over active edges $e = 1, \dots, m$. For each edge, it computes $d_e = \beta_{u_e} - \beta_{v_e}$ and $p_e = \text{clip}((\gamma w_e / \mu) d_e, -1, 1)$, then performs the endpoint updates

$$\Delta_f(:, u_e) \leftarrow \Delta_f(:, u_e) + \gamma w_e p_e, \quad \Delta_f(:, v_e) \leftarrow \Delta_f(:, v_e) - \gamma w_e p_e.$$

The accumulated matrix Δ_f is the fusion contribution added to the likelihood and sparsity gradients in the accelerated update.

Optimization algorithm. The implementation used in the experiments is the following active-edge APG procedure. Here T is the maximum number of APG iterations and ε is the stopping tolerance. It is the central exact L1 routine in `sparsefusion`.

Algorithm 2 Active-Edge Operator APG (operator)

Require: grouped data, edge list E , $\lambda, \gamma, \mu, T, \varepsilon$

- 1: Initialize $B^{(0)} = 0$, $W^{(0)} = 0$, $S^{(0)} = 0$
- 2: **for** $t = 1, \dots, T$ **do**
- 3: Compute groupwise likelihood gradient $\nabla \ell(B^{(t-1)})$
- 4: $A_s^* \leftarrow \text{clip}(\lambda B_s^{(t-1)} / \mu, -1, 1)$
- 5: Initialize fusion accumulator $\Delta_f \leftarrow 0$
- 6: **for** each edge $e = (u_e, v_e, w_e) \in E$ **do**
- 7: $d_e \leftarrow \beta_{u_e}^{(t-1)} - \beta_{v_e}^{(t-1)}$
- 8: $p_e \leftarrow \text{clip}((\gamma w_e / \mu) d_e, -1, 1)$
- 9: $\Delta_f(:, u_e) \leftarrow \Delta_f(:, u_e) + \gamma w_e p_e$
- 10: $\Delta_f(:, v_e) \leftarrow \Delta_f(:, v_e) - \gamma w_e p_e$
- 11: **end for**
- 12: $H_t \leftarrow \nabla \ell(B^{(t-1)}) + \lambda A_s^* + \Delta_f$
- 13: Compute L_U on active graph; update $\tilde{B}^{(t)}, S^{(t)}, W^{(t)}$
- 14: **if** $\|\tilde{B}^{(t)} - B^{(t-1)}\|_1 < \varepsilon p$ **then**
- 15: **break**
- 16: **end if**
- 17: $B^{(t)} \leftarrow \tilde{B}^{(t)}$
- 18: **end for**
- 19: Soft-threshold non-intercept rows by λ / n_g
- 20: **return** $\hat{B}$

4.2 L1 Chain Approximation

The chain approximation solver is a deliberately approximate method. It builds a depth-first traversal of a maximum spanning tree or of the thresholded graph, reorders the groups along that traversal, and keeps only the $k - 1$ adjacent chain edges. Let π denote the resulting group ordering and let $w = (w_1, \dots, w_{k-1})$ denote the adjacent chain weights after reordering. The fusion operator can then be computed with vectorized adjacent differences:

$$D_j = \beta_j - \beta_{j+1}, \quad j = 1, \dots, k - 1.$$

The vectorized implementation forms

$$D = B_{:,1:(k-1)} - B_{:,2:k}, \quad P = \text{clip}((\gamma/\mu)D \text{diag}(w), -1, 1),$$

and accumulates $\gamma P \text{diag}(w)$ on neighboring chain endpoints. This reduces the edge budget to the minimum needed to connect the groups. It does not optimize the same objective as the original graph unless the graph is already the selected chain. It is useful when the main objective is maximum speed and memory reduction and the user is willing to accept a small loss in graph fidelity.

Proposition 5 (Chain approximation target)

Let $G_\pi = (V, E_\pi)$ be the chain graph produced by the DFS/MST ordering, with edge weights inherited from the original graph. The chain approximation solver applies the exact active-edge L1 update to the smoothed fused objective on G_π . Define the original graph-fusion penalty as

$$R_G(B) = \gamma \sum_{(u,v) \in E} w_{uv} \|\beta_u - \beta_v\|_1,$$

and define $R_{G_\pi}(B)$ analogously on the chain. Then for any coefficient matrix B and any chain with $E_\pi \subseteq E$,

$$0 \leq R_G(B) - R_{G_\pi}(B) = \gamma \sum_{(u,v) \in E \setminus E_\pi} w_{uv} \|\beta_u - \beta_v\|_1.$$

Thus the approximation error is exactly the omitted-edge fusion penalty for the fitted coefficient matrix.

Proof After the groups are reordered by π , the vectorized adjacent-difference operator is the active-edge operator for the chain edges (π_j, π_{j+1}) , $j = 1, \dots, k-1$. The scatter updates therefore match the L1 active-edge update on G_π . If $E_\pi \subseteq E$, subtracting the chain penalty from the original graph penalty leaves precisely the sum over omitted edges, which is nonnegative. $\blacksquare$

Optimization algorithm. The chain approximation code first builds the DFS/MST order and adjacent chain weights, then runs a separate vectorized APG routine rather than calling the general edge loop. In Algorithm 3, $\eta = 1/L_U$ is the step size, H_t is the smoothed gradient at iteration t , $R^{(t)}$ is the accumulated scaled-gradient term, $Z^{(t)} = -R^{(t)}$ is the aggregate point used in the acceleration step, and $W^{(t)}$ is the accelerated iterate:

4.3 Active-Edge L2 Augmented Design

The L2 active-edge solver preserves the augmented-design strategy but changes how the design is built. Instead of allocating all pairwise fusion blocks and then assigning zero-weight rows, it finds the upper-triangular nonzero entries of G and builds sparse triplets only for those edges. As above, p_{eff} denotes the number of coefficient coordinates included in the L2 fusion block, and $n_{\text{fusion}} = p_{\text{eff}}m$ is the number of active fusion rows. The active-edge construction keeps the same augmented-design scaling: $\sqrt{\gamma(n + n_{\text{fusion}})}\sqrt{w_{gh}}$ multiplies the

Algorithm 3 Chain Approximation APG (`chain_approx`)

Require: grouped data, G , chain settings, $\lambda, \gamma, \mu, T, \varepsilon$

- 1: Build DFS/MST chain order π and adjacent chain weights $(w_1, \dots, w_{k-1})$
 - 2: Reorder groups by π ; initialize $B^{(0)} = 0$, $W^{(0)} = 0$, $R^{(0)} = 0$
 - 3: Compute chain Lipschitz bound L_U and set $\eta \leftarrow 1/L_U$
 - 4: **for** $t = 1, \dots, T$ **do**
 - 5: Compute groupwise likelihood gradient $\nabla \ell(B^{(t-1)})$
 - 6: $A_s^* \leftarrow \text{clip}(\lambda B_s^{(t-1)} / \mu, -1, 1)$
 - 7: $D \leftarrow B_{:,1:(k-1)}^{(t-1)} - B_{:,2:k}^{(t-1)}$
 - 8: $P \leftarrow \text{clip}\left(\frac{\gamma D \text{diag}(w)}{\mu}, -1, 1\right)$
 - 9: $C \leftarrow \gamma P \text{diag}(w)$
 - 10: Initialize fusion accumulator $\Delta_f \leftarrow 0$
 - 11: $\Delta_f(:, 1 : (k-1)) += C$, $\Delta_f(:, 2 : k) -= C$
 - 12: $H_t \leftarrow \nabla \ell(B^{(t-1)}) + \lambda A_s^* + \Delta_f$
 - 13: $\tilde{B}^{(t)} \leftarrow W^{(t-1)} - \eta H_t$
 - 14: $R^{(t)} \leftarrow R^{(t-1)} + \frac{t\eta}{2} H_t$
 - 15: $Z^{(t)} \leftarrow -R^{(t)}$
 - 16: $W^{(t)} \leftarrow (t\tilde{B}^{(t)} + 2Z^{(t)}) / (t+2)$
 - 17: **if** $\|\tilde{B}^{(t)} - B^{(t-1)}\|_1 < \varepsilon p$ **then**
 - 18: **break**
 - 19: **end if**
 - 20: $B^{(t)} \leftarrow \tilde{B}^{(t)}$
 - 21: **end for**
 - 22: Soft-threshold non-intercept rows by λ/n_g
 - 23: Reorder columns back to original group order
 - 24: **return** $\hat{B}$
-

positive and negative entries in the fusion rows. The final augmented design is passed to `glmnet` (Friedman et al., 2010) with the same lambda correction factor as the original wrapper.

This change leaves the fitted objective unchanged for a given active graph but reduces the number of constructed fusion rows from $p_{\text{eff}}k(k-1)/2$ to $p_{\text{eff}}m$. The largest expected gains occur when $m \ll k(k-1)/2$.

Optimization algorithm. The L2 path uses `glmnet` after building an augmented sparse regression problem. The fitted vector θ stacks the groupwise coefficients and is reshaped into the coefficient matrix B after fitting. Let p_B denote the number of coefficient rows in B after applying the intercept convention. The full-pairwise L2 reference is obtained from Algorithm 4 by replacing the active-edge loop over E with a loop over all $\binom{k}{2}$ group pairs, using zero weights for pairs absent from the graph.

Table 2 summarizes the asymptotic costs implied by the preceding solver descriptions. The purpose is to separate the common groupwise regression cost, represented by $O(np)$ terms, from the cost of the fusion representation. The L1 time entries describe one APG

Algorithm 4 L2 Augmented-Design Solver with `glmnet`

Require: grouped data (X, y, groups) , graph G , penalties λ, γ , intercept and loss-scaling conventions

- 1: Build block-diagonal design X_{block} over groups
 - 2: Extract active weighted edges $E = \{(u, v, w_{uv}) : u < v, G_{uv} = w_{uv} > 0\}$
 - 3: Let $\mathcal{P}$ be the coefficient coordinates included in L2 fusion, with $|\mathcal{P}| = p_{\text{eff}}$
 - 4: Initialize sparse triplet lists for X_{fusion} and set row counter $a \leftarrow 0$
 - 5: **for** each active edge $(u, v) \in E$ **do**
 - 6: **for** each coefficient coordinate $r \in \mathcal{P}$ **do**
 - 7: $a \leftarrow a + 1$
 - 8: Add triplet $(a, \text{col}(u, r), +\sqrt{w_{uv}})$
 - 9: Add triplet $(a, \text{col}(v, r), -\sqrt{w_{uv}})$
 - 10: **end for**
 - 11: **end for**
 - 12: Set $n_{\text{fusion}} \leftarrow a$ and materialize sparse matrix X_{fusion} from the triplets
 - 13: Scale fusion block by $\sqrt{\gamma(n + n_{\text{fusion}})}$
 - 14: Form augmented system $\tilde{X} = [X_{\text{block}}; X_{\text{fusion}}]$, $\tilde{y} = [y; 0]$
 - 15: Fit θ with `glmnet` on $(\tilde{X}, \tilde{y})$ using the corrected λ
 - 16: Reshape θ to $B \in \mathbb{R}^{p_B \times k}$
 - 17: **return** B
-

Table 2: Dominant representation costs of the solver variants. Here n is total sample size, p is feature count, p_{eff} is the number of coordinates included in the L2 fusion block, k is group count, $m = |E|$ is the number of active fusion edges, and $q = \binom{k}{2}$. The table isolates graph-representation costs and omits lower-order constants from R bookkeeping and solver overhead.

Method	Time	Memory
L1 full edge	$O(np + pkq)$	$O(np + pk + pq + kq)$
L1 active-edge operator	$O(np + pm)$	$O(np + pk + m)$
L1 chain approximation	$O(np + pk)$	$O(np + pk)$
L2 full pairwise	$O(np + p_{\text{eff}}q)$	$O(np + p_{\text{eff}}q)$
L2 active-edge	$O(np + p_{\text{eff}}m)$	$O(np + p_{\text{eff}}m)$

iteration, while the L2 time entries describe one pass over the augmented design. For L2, the table uses p_{eff} because the intercept convention determines how many coefficient coordinates are included in the fusion block. The active-edge L1 memory bound assumes the block processing described above, so edge-difference workspaces are not materialized for all m edges at once.

5 Experimental Design

5.1 Empirical Test Problems

The real-data experiments use public grouped-regression problems as optimization test cases. They provide different values of n , p , k , and graph sparsity while keeping the statistical objective fixed across solver representations. This makes them useful for testing the representation question studied in the paper, since the relative cost of full pairwise construction and active-edge construction changes as the number of groups and graph edges change. The reported figures use ten datasets with five outer folds for each dataset. Across these datasets, the number of observations ranges from 630 to 200,000, the number of features from 3 to 100, and the number of groups from 43 to 229. Most of the real-data test problems are drawn from Rdatasets mirrors (Arel-Bundock, 2026). The remaining sources are UCI datasets (Redmond, 2002; Trindade, 2015), Our World in Data CO2 data (Ritchie et al., 2023), World Bank indicators (World Bank, 2026), and package datasets (Pinheiro and Bates, 2000; Raudenbush and Bryk, 2002). The exact dataset list, preprocessing scripts, and fold assignments are provided with the replication materials.

For the side-by-side L1 and L2 accuracy and timing figures, we highlight three representative datasets rather than plotting every dataset in the main panel. `communities_crime` gives a higher-feature problem with a moderate number of groups. `hsb82` and `owid_co2` give many-group problems with smaller feature counts, where the cost difference between all-pair construction and active-edge construction is easier to see. These three datasets are used for the detailed accuracy and timing panels, while the runtime-memory Pareto plots summarize the broader ten-dataset real-data set.

5.2 Evaluation Protocol

All reported real-data results use grouped outer cross-validation with five stratified folds and random seed 20260221 (Kohavi, 1995; Varma and Simon, 2006). Splits are made at the observation level while preserving group representation where possible; datasets with very small groups are filtered using a minimum group size of seven. All graph construction, hyperparameter selection, and standardization are performed inside the outer-training fold so that the held-out test fold is not used in model selection.

For each outer fold, a baseline sparsity penalty is selected with fusion disabled, $\gamma = 0$, on the outer-training data by `cv.glmnet` with three inner folds and `lambda.min`. We denote this selected sparsity penalty by λ in Equation 1. Holding this λ fixed, a separate inner train-validation split of the outer-training data selects the fusion penalty γ in Equation 1 by validation root mean squared error (RMSE). This simple tuning protocol is deliberate: the goal of the experiments is to compare solver representations under a common protocol, not to optimize hyperparameter-search efficiency. It was sufficient to select nontrivial fused models that outperform the featureless baseline in the reported benchmark panels. The final model is then refit on the full outer-training fold using the selected λ and γ and evaluated on the held-out outer fold. The featureless baseline predicts the training-fold mean for the corresponding response. Runtime refers to model fitting only, and memory is reported as the peak resident set size of the fitting process. The computational experiments were run on Northern Arizona University’s Monsoon computing cluster (Northern Arizona University

Advanced Research Computing, 2026), with solver variants in each comparison evaluated under the same software environment and job configuration. Timing experiments were submitted as Slurm array jobs with one CPU core per task and a 24-hour time limit. Jobs were split into small, medium, and large memory tiers and requested 16 GB, 32 GB, and 64 GB of memory, respectively. The tier assignment used a deterministic workload score based on n , p , and k , with an additional multiplier for the full-pairwise reference methods. The archived job scripts do not request whole-node exclusive allocation or pin jobs to a particular Monsoon node generation, so absolute wall-clock times may reflect normal shared-cluster and hardware variability. For this reason, timing results are interpreted primarily as relative comparisons between solver representations run under the same scheduling protocol.

5.3 Graph Construction and Hyperparameters

The real-data benchmark constructs a data-driven group graph from training data, following the use of graph-based relationships and k -nearest-neighbor fusion graphs in prior work (Hallac et al., 2015; Padilla et al., 2020). Group centroids are computed on the outer-training fold, a k -nearest-neighbor graph is formed from centroid distances, inverse-distance weights are used, and connectivity is enforced by adding nearest inter-component links. The manifest records $k = 5$ nearest neighbors, inverse-distance weights, and MST-style connectivity augmentation. This follows the general idea that graph structure should encode similarity between groups rather than defaulting to a complete graph.

Each dataset is evaluated with five outer folds. The regularization parameter λ is warm-started from `cv.glmnet`; the fusion parameter γ is selected from the grid

$$10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}.$$

The L1 optimization tolerance is 10^{-5} with a maximum of 1200 iterations in the HPC manifest. Runs use intercepts for L1 models, no feature scaling inside the L1 solver, standardized inputs for benchmark construction, an edge-block size of 256 for the operator implementation, and the adaptive operator fallback threshold recorded in the manifest. The L2 panel uses the same outer-fold and graph protocol with the L2 solvers.

5.4 Compared Methods

For L1 fusion, the benchmark compares the legacy full construction (`old_l1`), the exact active-edge operator solver (`operator`), the chain approximation (`chain_approx`), and a featureless baseline. For L2 fusion, the benchmark compares the legacy augmented design (`old_l2`), the active-edge augmented design (`new_l2`), and the featureless baseline.

The exact comparisons are intentionally conservative. The operator and active-edge L2 methods are compared against legacy implementations using the same graph, outer folds, selected hyperparameters, and prediction pipeline. Thus the reported differences isolate representation and construction costs, not changes in the statistical model or optimizer family. Comparisons against ADMM, split-Bregman, or other edge-splitting optimizers answer a broader question about optimizer choice; the present experiments instead test whether the sparse graph can be preserved while keeping the L1 solver within the smoothed proximal-gradient family and the L2 solver within the augmented-design plus `glmnet` family.

The chain approximation L1 method is reported separately because it changes the graph and is therefore an approximation rather than an exact replacement.

6 Results

The results are organized around two questions. The first experiment illustrates when graph fusion is statistically useful relative to unfused `glmnet` baselines. The remaining results evaluate the main optimization question of the paper, namely whether sparse-graph representations preserve predictive accuracy while reducing runtime and memory for L1 and L2 grouped fused regression.

6.1 Changepoint Structure Relative to Lasso

Before comparing solver representations, we first show a setting where fusion is statistically useful relative to unfused lasso baselines. The purpose of this experiment is not to measure scalability, but to make clear why a graph-fused regression model is useful in the first place. The data-generating process has ordered groups whose regression coefficients are neither globally identical nor unrelated across groups. The intended structure is piecewise constant across three contiguous blocks: groups 1–20, groups 21–40, and groups 41–60 each have stable coefficients, with changes only at the two boundaries between these blocks.

We generated an ordered-group regression problem with $k = 60$ groups and $p = 20$ predictors. The true coefficient paths are piecewise constant: groups 1–20 share one coefficient block, groups 21–40 share a second block, and groups 41–60 share a third block. Thus the two true changepoints occur after groups 20 and 40. Each group has 25 training observations for fitting, 10 validation observations for tuning, and 80 test observations. The fusion graph is the chain over the ordered groups, so the fused model penalizes differences only between neighboring groups.

We compare three estimators. The lasso(all) baseline fits one shared coefficient vector using all groups together, so it is stable but cannot represent changepoints across groups. The lasso(same) baseline expands the design with group-feature interactions and fits separate group coefficients without any fusion penalty; it can represent differences across groups, but it does not borrow strength across neighboring groups. The fused lasso model also estimates coefficient vectors for each group, but adds an L1 fusion penalty on adjacent groups in the chain so that it optimizes the trade-off between sharing and allowing coefficients to differ across groups. Both lasso baselines are fit with `glmnet`. Their lasso penalties are selected by five-fold `cv.glmnet` on the training data, while the fused lasso selects (λ, γ) by validation RMSE over a fixed grid.

Figure 4 shows the fitted path for the first predictor across the 60 ordered groups. The lasso(all) path is nearly flat because it must use the same coefficient vector for all groups. The lasso(same) path varies by group, but without fusion it produces a noisy path with many spurious jumps. The fused lasso path is piecewise constant and changes at the two true changepoints. Numerically, the fused lasso obtains the lowest test RMSE (0.708), the lowest relative coefficient error (0.079), and exactly two large estimated jumps, both at the true changepoints. By comparison, lasso(all) has test RMSE 1.957 and relative coefficient error 0.848, while lasso(same) improves prediction error but produces 59 large jumps.

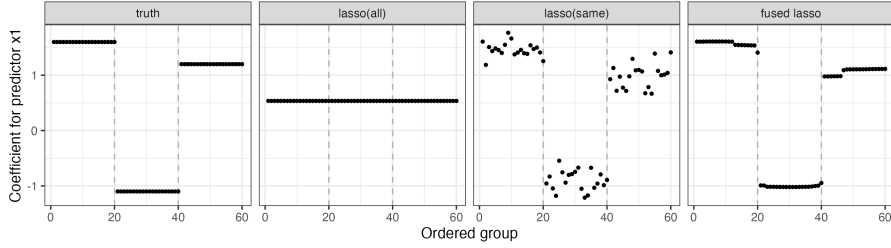


Figure 4: Controlled changepoint example comparing fused regression with unfused lasso baselines. Points show the coefficient for the first predictor across all 60 ordered groups, and dashed vertical lines mark the true changepoints. `lasso(all)` fits only one shared coefficient path, `lasso(same)` produces noisy separate paths, and the fused lasso model recovers the piecewise-constant coefficient structure. The lasso baselines are fit with `glmnet`.

6.2 Accuracy Preservation on Representative Datasets

Figure 5 compares prediction accuracy for L1 and L2 fusion on three representative datasets. These datasets span a high-feature setting with fewer groups (`communities_crime`) and two many-group settings with moderate feature counts (`hsb82` and `owid_co2`). The figure is intended to isolate the accuracy question: does replacing all-pair construction with active-edge construction change the test error?

For the exact sparse-graph solvers, the answer is no to numerical precision. The L1 active-edge operator overlaps the full-edge L1 reference on all three datasets, and the L2 active-edge augmented design similarly overlaps the full-pairwise L2 reference. This matches the equivalence results in Section 4: the active-edge representation changes the data structure used by the solver, not the fitted objective. The chain approximation L1 method is shown separately in the L1 row because it is an approximation; its accuracy can differ because replacing the graph by a chain changes the fusion penalty.

6.3 Timing Gains on Representative Datasets

Figure 6 reports the corresponding fit times for the same three datasets and solver families. The timing figure addresses the second part of the empirical claim: once accuracy is preserved, the benefit of the active-edge representation is computational. The L1 operator avoids applying fusion updates to inactive graph pairs, while the L2 builder avoids materializing zero-weight augmented rows before calling `glmnet`.

The number of groups is the main driver of this efficiency gap. A full pairwise construction has $q = \binom{k}{2}$ candidate group pairs, so increasing k increases the graph side of the computation quadratically even when the intended graph is sparse. In contrast, the active-edge representation scales with the number of nonzero graph edges m . For sparse graphs such as nearest-neighbor or chain-like graphs, m grows much more slowly than $\binom{k}{2}$; for a chain, $m = k - 1$. This is why the high- k datasets `hsb82` and `owid_co2` show the clearest timing separation: the full construction still pays for many absent group pairs, whereas the active-edge solvers only process the edges present in the graph.

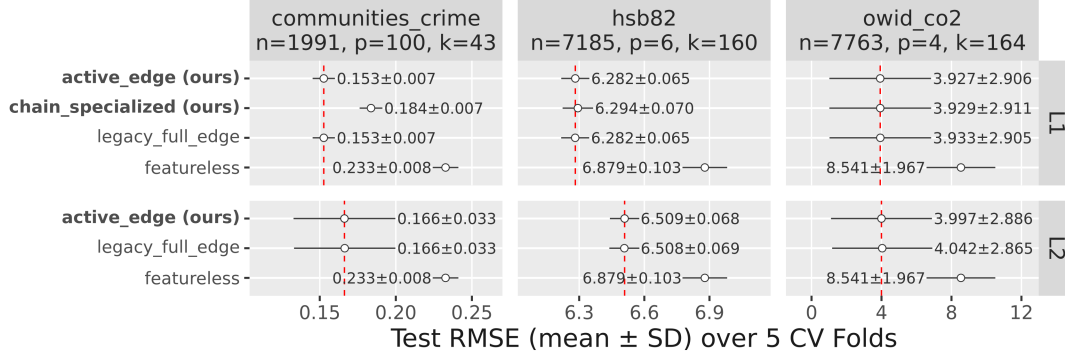


Figure 5: Prediction accuracy for L1 and L2 fusion on representative datasets. Points show test RMSE means over five cross-validation folds, with horizontal bars showing one standard deviation. The label `legacy_full_edge` denotes the full-construction reference. Exact active-edge solvers preserve the full-construction RMSE, while the chain approximation L1 method is an approximate variant.

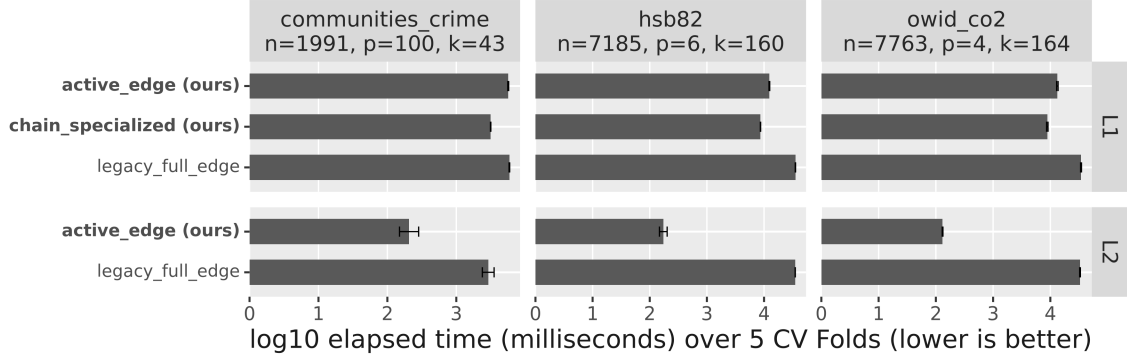


Figure 6: Fit time for the same datasets and solver families shown in Figure 5. Bars show mean $\log_{10}$ elapsed time in milliseconds over five cross-validation folds, with error bars showing one standard deviation. Lower values are better.

The L2 timing gains are especially large because all-pair augmented-design construction is expensive when k is large and the graph is sparse. The L1 operator also improves timing, although the gain is smaller because the smoothed proximal-gradient iterations retain fixed overhead beyond graph updates. The chain approximation L1 method is fastest among the L1 variants in these examples, but it should be interpreted as an accuracy-efficiency tradeoff rather than an exact replacement for the sparse graph.

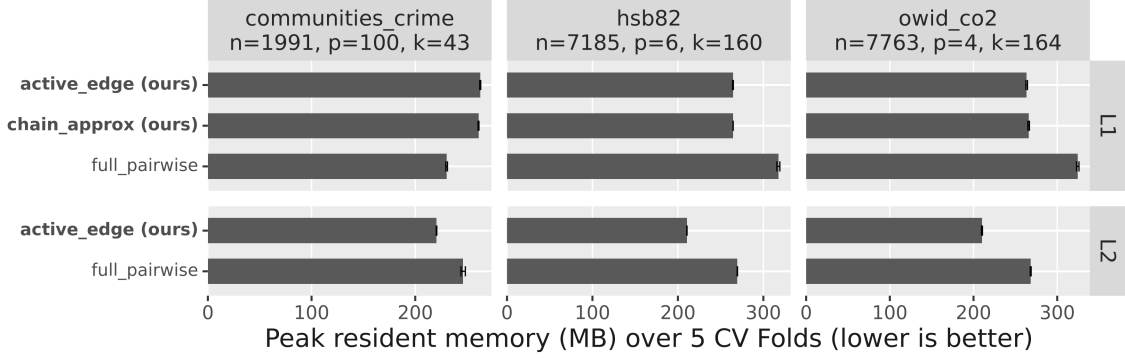


Figure 7: Peak resident memory for the same representative datasets and solver families shown in Figure 5. Bars show mean peak resident set size in MB over five cross-validation folds, with error bars showing one standard deviation. Lower values are better. The figure reports absolute memory usage, complementing the relative memory ratios in the Pareto plots.

6.4 Memory Gains on Representative Datasets

Figure 7 reports the corresponding absolute peak resident memory. This complements the relative memory ratios reported later because the representative fits use hundreds of megabytes rather than many gigabytes of memory. This matters for interpreting the optimization results: sparse construction reduces the graph objects created by the solver, but peak resident memory also includes the design matrix, response vector, group indices, coefficient iterates, validation state, and R runtime allocations.

For L2 fusion, the active-edge augmented design consistently reduces peak resident memory relative to the full-pairwise reference. The reason is direct: the active-edge builder avoids materializing augmented rows for absent graph edges, so it holds fewer sparse-matrix entries, row indices, and construction triplets before calling `glmnet`. For L1 fusion, the memory gains are smaller and less uniform. The active-edge operator reduces the working set for fusion differences and scatter updates, but the smoothed proximal-gradient state and fixed data objects remain. The chain approximation is included in this panel because it further reduces the graph object, although it changes the fusion penalty.

6.5 Runtime-Memory Pareto Gains

Figure 8 summarizes the runtime-memory tradeoff for the exact active-edge solvers on the real-data benchmark panel. Each point compares an active-edge solver with its full-pairwise reference on the same dataset. The horizontal coordinate is the log ratio of fit time, and the vertical coordinate is the log ratio of peak memory. Thus points to the left of zero are faster than the reference, points below zero use less memory, and points in the lower-left quadrant improve both. Labels report the dataset, n , p , k , and the percent RMSE change relative to the reference.

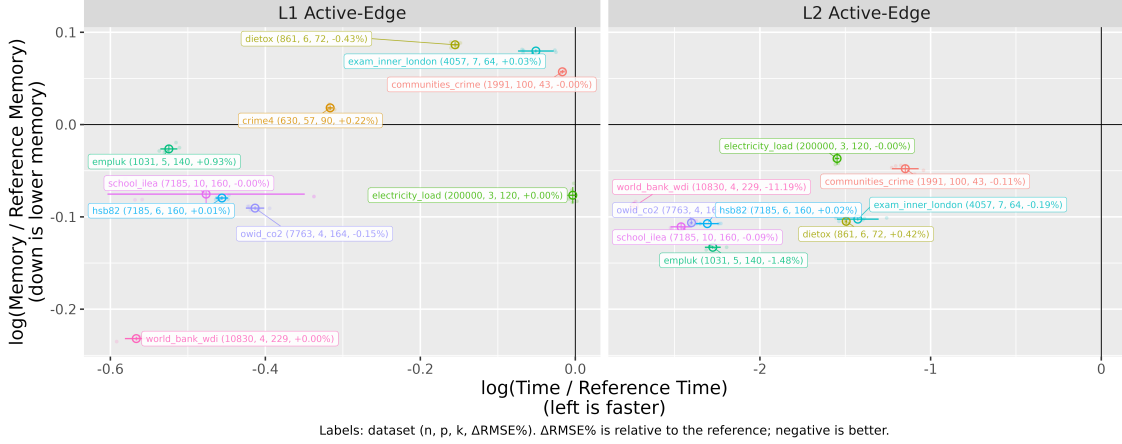


Figure 8: Runtime-memory Pareto plots for the exact active-edge solvers. The L1 panel compares the active-edge operator with the full-pairwise L1 reference, and the L2 panel compares active-edge augmented-design construction with the full-pairwise L2 reference. Left is faster, down is lower peak memory, and the lower-left quadrant indicates improvement on both efficiency axes.

We use the plot in the standard multiobjective sense of a Pareto comparison: a method is preferable only if it improves one resource without worsening the other, or if the tradeoff is explicit (Miettinen, 1999). This is more informative than reporting runtime and memory separately because an optimization change can improve one resource while leaving the other unchanged, or even slightly worse. The RMSE labels stay near zero in both panels, so the figure is primarily an efficiency comparison rather than an accuracy comparison.

The two panels connect the separate timing and memory results. Most L1 datasets move left, and many also move downward, but the memory gains are smaller and less uniform than the timing gains. The L2 panel is shifted farther left and mostly downward because active-edge augmented-design construction reduces both the number of rows built before `glmnet` and the sparse objects held by the worker process. These patterns match the construction-cost analysis in Section 4.

The memory axis should be interpreted as an allocation and working-set comparison within moderate-memory jobs. Across the L1 benchmark, peak resident memory ranged from about 72 MB for featureless fits to 452 MB for the largest full-pairwise reference fit; the active-edge operator and chain approximation had median peak resident memories of about 265 MB. Across the L2 benchmark, peak resident memory ranged from about 71 MB for featureless fits to 439 MB overall, with median peak resident memory about 204 MB for active-edge L2 and 264 MB for the full-pairwise L2 reference. Some timing gains may therefore also reflect improved cache locality from avoiding full-pairwise materialization, in addition to the reduced arithmetic and allocation work.

Table 3: Summary of exact sparse-graph efficiency gains. Speedup is computed relative to the corresponding full-construction reference; memory reduction is the decrease in peak resident memory.

Solver comparison	Accuracy change	Median speedup	Median memory reduction
L1 active-edge operator vs. full-pairwise L1	maximum mean-RMSE difference $< 10^{-5}\%$	2.26x	10.7%
L2 active-edge augmented design vs. full-pairwise L2	-0.025% median RMSE difference	14.9x	19.9%

6.6 Synthetic Scaling Experiments

The repository also includes synthetic scaling experiments generated by the `atime` benchmark scripts. These experiments isolate solver scaling under controlled changes to the number of groups and graph structure. They support the same qualitative conclusion as the real-data benchmarks: avoiding all-pair graph construction matters most when k grows and the graph remains sparse.

7 Discussion

The experiments show that sparse fusion graphs need corresponding sparse computational representations. When the graph has m edges, an implementation that constructs $O(k^2)$ pairwise constraints can lose the computational advantage of using a sparse graph. This is a general issue for edge-penalized regression, not only for `sparsefusion`. Any method whose penalty decomposes over graph edges can pay an unnecessary quadratic cost if the graph is represented by a complete set of candidate pairs. The L1 operator and L2 active-edge builders recover the intended $O(pm)$ fusion representation while preserving the original objective in the exact cases.

The exact L1 operator is the safest default among the L1 variants. It preserves the same fitted objective as the legacy active-edge formulation and gives substantial median runtime and memory improvements on the completed benchmark summary. The chain approximation solver is more aggressive. It is best viewed as an approximation for exploratory or large-scale settings where a small loss in predictive accuracy is acceptable.

The solver representation aligns the computational graph with the modeling graph. Edge-explicit L1 fusion updates, blocked edge processing, DFS/MST chain approximation, and active-edge L2 matrix construction are instances of that representation. Proposition 1 gives the corresponding smoothed-objective convergence guarantee for the L1 APG updates, while Propositions 2 and 3 identify the exact cases where the new representations preserve the legacy objective. Proposition 4 makes the representation-dependent graph cost explicit. The chain approximation method is separated from those exact methods because replacing a graph by a chain changes the fusion geometry; Proposition 5 states the precise chain objective being optimized and the omitted-edge penalty that accounts for the approximation.

The empirical evidence is strongest for the exact L1 operator and active-edge L2 construction. The chain variant is included because it is a useful solver mode in the package, but it involves an additional graph-approximation choice. For that reason, the recommended

default for sparse L1 fusion is the exact active-edge operator, with chain approximation fitting reserved for exploratory or memory-constrained settings.

7.1 When Sparse-Graph Representations Help

The sparse-graph representation is most useful when the modeling graph has far fewer active edges than the complete graph. The estimators considered in this paper allow arbitrary fusion graphs, including complete graphs. If the intended graph is complete, then $m = \binom{k}{2}$ and active-edge construction remains correct but does not reduce the asymptotic graph-representation cost. The computational gains therefore come from settings where the graph encodes a structured notion of similarity, such as temporal adjacency, spatial neighborhoods, nearest-neighbor group similarity, or task relationships. For complete or nearly complete graphs, the only L1 variant in this paper that can still reduce the fusion edge budget substantially is the chain approximation approximation. It does so by replacing the original graph with a connected chain, so the gain comes from changing the optimization target rather than from an exact sparse representation of the same graph. The empirical results therefore treat the chain method separately: it can be much faster, but it may sacrifice some prediction accuracy because it omits many original fusion edges.

This distinction is separate from the statistical question of which graph is best for prediction. A complete graph may be appropriate when all group pairs should be eligible for fusion. In contrast, sparse graphs are appropriate when domain knowledge or training-fold similarity suggests that only a limited set of group relationships should be penalized directly. The contribution of the active-edge solvers is to make this sparse modeling choice visible to the optimizer: when the graph is sparse, computation and memory scale with m rather than with all $\binom{k}{2}$ possible group pairs.

7.2 Scope of the Optimization Comparisons

The experiments intentionally compare the new representations against the legacy grouped fused-regression solvers rather than against a broad collection of alternative optimizer families. This choice is central to the empirical design. The L1 legacy solver and the exact active-edge operator both use the same smoothed proximal-gradient framework; the L2 legacy and active-edge solvers both use the same augmented-design reduction followed by `glmnet`. The comparisons therefore hold fixed the statistical objective, graph, folds, hyperparameter selection, prediction pipeline, and optimizer family. The primary changing factor is the representation of the graph-fusion term: dense or all-pair construction versus active-edge construction. This isolation is what makes the empirical results interpretable as evidence for the sparse-graph representation principle.

Alternative optimizers, including ADMM and split-Bregman methods, are important for fused-lasso problems and can be very effective in other settings. They answer a different question. An ADMM formulation would typically introduce auxiliary variables for edge differences, enforce $z_{uv} = \beta_u - \beta_v$ with augmented-Lagrangian updates, alternate between coefficient, edge-variable, and dual updates, and require choices of penalty parameters and stopping rules. Such a comparison would mix representation effects with optimizer effects: linear-system solves or preconditioners, penalty-parameter tuning, convergence tolerances, warm starts, sparse-matrix backends, and implementation language could all influence run-

time and memory. Those factors are scientifically interesting, but they would not isolate the question studied here.

This distinction also clarifies the role of the legacy fused-lasso baseline. It is not included because it is the strongest possible optimizer for every graph-fused problem. It is included because it is the direct predecessor for the grouped fused-regression objective and software interface considered in this paper. Beating that baseline while preserving the same optimizer family shows that the gain comes from avoiding zero-weight all-pair graph construction rather than from replacing the optimization architecture. The same active-edge principle could be incorporated into ADMM or other edge-splitting methods, where the number of auxiliary edge variables and dual updates would also scale with the represented edge set. Developing and tuning such solvers is a separate algorithmic comparison and is left for future work.

There are three important limitations. First, the guarantees are algorithmic: they cover representation equivalence, smoothing error, convergence for the smoothed L1 objective, and graph-construction complexity. The statistical properties are those of the underlying fused-lasso and graph-guided regression objectives, while the contribution here is to preserve those objectives with a sparser computational representation. Second, the largest gains occur when the user-supplied or data-derived graph is sparse. If the modeling graph is close to complete, the active-edge representation has little edge-count advantage. Third, the implementation used in the experiments is written in R, so small problems can be dominated by fixed interpreter, allocation, and `glmnet` overheads. These limits explain the occasional small-benchmark slowdown and motivate the recommendation to use the exact operator as the default sparse-graph path rather than as a universal speed guarantee.

8 Implementation Details

The main implementation challenge was to change the graph representation without changing the fitted objective. In the original all-pair construction, inactive graph pairs can still appear as zero-weight algebraic objects. This is convenient for indexing, but it hides sparsity from the solver. The L1 operator code therefore first converts the fusion graph into edge arrays $(u_e, v_e, w_e)_{e=1}^m$ and keeps those arrays as the only fusion structure used inside the accelerated iterations. Each edge update computes a coefficient difference, clips the smoothed dual quantity, and scatters the result back to the two incident group columns with opposite signs. The difficult part was not the edge formula itself, but preserving the same orientation, weighting, intercept convention, group ordering, and final thresholding semantics as the reference implementation.

Memory management was another practical issue. A direct vectorized implementation can form a large edge by coefficient matrix during each L1 iteration. That is fast for moderate problems but can dominate memory when both p and m are large. The implementation therefore supports blocked edge processing, which applies the same scatter updates in edge batches while keeping the accumulated gradient in the original coefficient matrix shape. The chain solver required a separate bookkeeping step: groups are temporarily reordered along the selected chain for fast adjacent differences and then reordered back before returning fitted coefficients.

The L2 implementation required a different construction. The statistical problem is represented as an augmented least-squares design passed to `glmnet`. The key coding step was to build this augmented matrix directly from sparse triplets. For each active edge (g, h) and each fused coefficient coordinate, the builder emits one row with opposite signed entries in the two group blocks and a zero augmented response. This avoids allocating rows for inactive group pairs. The implementation also preserves the original loss scaling and lambda correction convention, so that any observed difference between full-pair and active-edge runs comes from the represented graph rows rather than from a changed fitting convention.

The package retains Rcpp/C++ routines from the original L1 implementation, but the active-edge L1 operator and L1 chain approximation used for the sparse-graph comparisons are implemented in R. The L2 sparse-graph solver builds the sparse augmented design in R and delegates the regularized path computation to `glmnet`.

9 Software Availability

The algorithms are available in the open-source R package `sparsefusion` at <https://github.com/EngineerDanny/sparsefusion>. A grouped regression problem is supplied as a feature matrix X , response vector y , group labels `groups`, and a symmetric fusion graph matrix G . For new data, the recommended entry point is `sparse_fusion`. The argument `fusion` selects l1 or l2 fusion, and the argument `solver` selects `active_edge`, `full_pairwise`, or `chain_approx`. The `active_edge` solver is the exact sparse graph representation for L1 and L2 fusion, `full_pairwise` runs the reference all-pair construction, and `chain_approx` runs the approximate L1 chain solver. The package also retains the lower-level fitting functions used by the experiments for direct solver comparisons.

For example, an exact sparse-graph L1 fit on new grouped data can be run as follows:

```
library(sparsefusion)

fit <- sparse_fusion(
  X, y, groups, G,
  lambda = 1e-3,
  gamma = 1e-2,
  fusion = "l1",
  solver = "active_edge"
)
```

The returned object from each fitting function is a coefficient matrix with one column per group, so prediction on new observations requires selecting the coefficient column associated with the observation's group. The package includes examples, benchmark scripts, and a minimal reviewer example in the repository README. The paper experiments call the same package functions used for new data analysis, with fixed folds and graph construction scripts stored in the repository.

10 Reproducibility

The algorithms are implemented in the `sparsefusion` R package, version 0.1.0, released under the MIT license. The project repository and issue tracker are available at <https://github.com/EngineerDanny/sparsefusion>. The experiments in this paper were evaluated from repository commit prefix `99c69ae`. The benchmark environment used R version 4.5.2 (2025-10-31). The package contains R and Rcpp/C++ code, links against the BLAS and LAPACK libraries exposed through R, and uses `glmnet`, `Matrix`, `Rcpp`, and `RcppEigen` for compiled numerical routines used in the timing experiments. The saved job metadata do not record the compiler version, BLAS/LAPACK implementation, operating-system release, or physical node model. Consequently, the response-time results should be read as within-environment relative comparisons between solver representations rather than as hardware-normalized absolute benchmarks. The package includes documentation, a vignette, and `testthat` coverage for the solver routines used in the experiments. The large-scale empirical experiments were run on Northern Arizona University’s Monsoon computing cluster (Northern Arizona University Advanced Research Computing, 2026) using fixed job settings within each solver comparison. The Slurm arrays requested one CPU core per task and used three memory tiers: 16 GB for small jobs, 32 GB for medium jobs, and 64 GB for large jobs. Tiering was based on $s = np + pk + k^2$, with s doubled for `old_l1` and `old_l2` because those full-pairwise reference methods are expected to require more memory. The L1 run used approximate tier cutoffs $s \leq 24,196$ for small jobs, $24,196 < s \leq 99,050$ for medium jobs, and $s > 99,050$ for large jobs; the L2 run used the same rule with nearly identical empirical quantile cutoffs. The L1 benchmark arrays correspond to Slurm job IDs 27073874–27073876, and the L2 benchmark arrays correspond to Slurm job IDs 27086933–27086935. The archived Slurm script does not include an exclusive-node request, and the saved result files record the cluster (`monsoon`) and job IDs but not the physical node model.

The real-data benchmark summaries used in the figures are `build/hpc/final_test_summary.csv` for the L1 panel and `build/hpc/l2_knn/final_test_summary.csv` for the L2 panel. The structured synthetic example is generated by the changepoint benchmark script under `scripts`. The manifest-based benchmark scripts are stored under `scripts/hpc`, and the paper source is `publication/paper1/sparsefusion_jmlr.tex`.

11 Conclusion

This paper studied the computational representation of sparse fusion graphs in grouped fused regression. The main finding is that a sparse statistical graph should remain sparse inside the optimizer. For L1 fusion, the active-edge operator evaluates the same smoothed fusion gradient as the full-edge construction after zero-weight pairs are removed. For L2 fusion, the active-edge augmented design gives the same quadratic fusion objective while constructing rows only for nonzero graph edges. These exact representations change the dominant fusion cost from all-pairs construction to construction that scales with the represented edge set.

The empirical results support the same conclusion. The exact L1 and L2 active-edge solvers preserve predictive accuracy relative to the corresponding full-construction references while reducing fit time and, in many cases, peak memory. The chain approximation L1 solver

gives an additional approximate option when reducing the edge budget is more important than preserving the original graph exactly. More broadly, graph-penalized regression methods should carry the edge structure of the penalty into the numerical representation rather than encode a sparse graph through dense all-pairs objects.

Several extensions are natural. First, the same active-edge representation principle could be incorporated into other optimizer families, including ADMM and split-Bregman methods, where auxiliary variables and dual updates should also scale with the represented edge set. Second, adaptive graph construction and hyperparameter selection could be studied jointly with the sparse solver representation, since the statistical value and computational cost of fusion both depend on the chosen graph. Third, lower-level sparse matrix and compiled implementations could further separate the algorithmic savings from fixed runtime overheads in R. These directions would test how far the representation principle extends beyond the grouped fused-regression solvers considered here.

Acknowledgments and Disclosure of Funding

The authors thank the maintainers of the public datasets and R packages used in the empirical evaluation. Some of the computational analyses were run on Northern Arizona University’s Monsoon computing cluster, funded by Arizona’s Technology and Research Initiative Fund. This work was supported by National Science Foundation award 2125088 through the Rules of Life program. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

References

- Vincent Arel-Bundock. Rdatasets: A collection of datasets originally distributed in R packages, 2026. URL <https://vincentarelbundock.github.io/Rdatasets/>.
- Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.
- Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011. doi: 10.1561/22000000016.
- Xi Chen, Qihang Lin, Seyoung Kim, Jaime G. Carbonell, and Eric P. Xing. Graph-structured multi-task regression and an efficient optimization method for general fused lasso. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1–10, 2010.
- Xi Chen, Qihang Lin, Seyoung Kim, Jaime G. Carbonell, and Eric P. Xing. Smoothing proximal gradient method for general structured sparse regression. *The Annals of Applied Statistics*, 6(2):719–752, 2012a.

- Xiaohui Chen, Xinghua Shi, Xing Xu, Zhiyong Wang, Ryan Mills, Charles Lee, and Jinbo Xu. A two-graph guided multi-task lasso approach for eQTL mapping. In *Proceedings of the Fifteenth International Conference on Artificial Intelligence and Statistics*, volume 22 of *Proceedings of Machine Learning Research*, pages 208–217, 2012b.
- Frank Dondelinger, Sach Mukherjee, and The Alzheimer’s Disease Neuroimaging Initiative. The joint lasso: High-dimensional regression for group structured data. *Biostatistics*, 21(2):219–235, 2020. doi: 10.1093/biostatistics/kxy035.
- Jerome Friedman, Trevor Hastie, and Robert Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010.
- David Hallac, Jure Leskovec, and Stephen Boyd. Network lasso: Clustering and optimization in large graphs. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 387–396, 2015.
- Holger Hoefling. A path algorithm for the fused lasso signal approximator. *Journal of Computational and Graphical Statistics*, 19(4):984–1006, 2010.
- Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, pages 1137–1143. Morgan Kaufmann, 1995.
- Kaisa Miettinen. *Nonlinear Multiobjective Optimization*. Kluwer Academic Publishers, Boston, 1999. doi: 10.1007/978-1-4615-5563-6.
- Yurii Nesterov. Smooth minimization of non-smooth functions. *Mathematical Programming*, 103(1):127–152, 2005.
- Northern Arizona University Advanced Research Computing. Overview of monsoon high-performance computing resources, 2026. URL <https://in.nau.edu/arc/overview/>. Accessed 2026-05-28.
- Nooshin Omranian, Jeanne M. O. Eloundou-Mbebi, Bernd Mueller-Roeber, and Zoran Nikoloski. Gene regulatory network inference using fused LASSO on multiple data sets. *Scientific Reports*, 6:20533, 2016. doi: 10.1038/srep20533.
- Oscar Hernan Madrid Padilla, James Sharpnack, James G. Scott, and Ryan J. Tibshirani. Adaptive nonparametric regression with the K-nearest neighbour fused lasso. *Biometrika*, 107(2):293–310, 2020.
- Ashley Petersen, Daniela Witten, and Noah Simon. Fused lasso additive model. *Journal of Computational and Graphical Statistics*, 25(4):1005–1025, 2016.
- José C. Pinheiro and Douglas M. Bates. *Mixed-Effects Models in S and S-PLUS*. Springer, New York, 2000. doi: 10.1007/b98882.
- Stephen W. Raudenbush and Anthony S. Bryk. *Hierarchical Linear Models: Applications and Data Analysis Methods*. Sage Publications, Thousand Oaks, CA, 2 edition, 2002.

- Michael Redmond. Communities and crime. UCI Machine Learning Repository, 2002. DOI: 10.24432/C53W3X.
- Hannah Ritchie, Pablo Rosado, and Max Roser. CO2 and greenhouse gas emissions. Our World in Data, 2023.
- Wesley Tansey, Yu-Xiang Wang, David M. Blei, and Raul Rabadan. The DFS fused lasso: Linear-time denoising over general graphs. *Journal of Machine Learning Research*, 19(176):1–36, 2018.
- Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996.
- Robert Tibshirani, Michael Saunders, Saharon Rosset, Ji Zhu, and Keith Knight. Sparsity and smoothness via the fused lasso. *Journal of the Royal Statistical Society: Series B*, 67(1):91–108, 2005.
- Ryan J. Tibshirani and Jonathan Taylor. The solution path of the generalized lasso. *The Annals of Statistics*, 39(3):1335–1371, 2011.
- Artur Trindade. Electricityloadaddiagrams20112014. UCI Machine Learning Repository, 2015. DOI: 10.24432/C58C86.
- Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006. doi: 10.1186/1471-2105-7-91.
- Yu-Xiang Wang, James Sharpnack, Alexander J. Smola, and Ryan J. Tibshirani. Trend filtering on graphs. *Journal of Machine Learning Research*, 17(105):1–41, 2016.
- World Bank. World development indicators. World Bank DataBank, 2026.
- Gui-Bo Ye and Xiaohui Xie. Split bregman method for large scale fused lasso. *Computational Statistics & Data Analysis*, 55(4):1552–1569, 2011. doi: 10.1016/j.csda.2010.10.021.
- Feng Yu, Archer Yi Yang, and Teng Zhang. A graph decomposition-based approach for the graph-fused lasso. *Journal of Statistical Planning and Inference*, 235:106221, 2025. doi: 10.1016/j.jspi.2024.106221.
- Ming Yuan and Yi Lin. Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society: Series B*, 68(1):49–67, 2006.
- Hui Zou and Trevor Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2):301–320, 2005. doi: 10.1111/j.1467-9868.2005.00503.x.